ARTIFICIAL INTELLIGENCE LAB

CS 2005 | Final Project Report

Baymax

AI-Powered Career Coaching System for CS Students in Pakistan

COURSE	SUBMISSION	INSTITUTION
CS 2005 — AI	**Spring 2026**	**FAST NUCES KHI**

GROUP MEMBERS	Taha Zaidi	Amna Khan	Kissa Zehra	Aiza Gazyani

1. Problem Statement

Pakistan produces over **500,000 STEM graduates annually**, yet more than **60% of CS graduates face unemployment or underemployment** despite holding technical degrees. The root causes are consistent and addressable: resumes fail Applicant Tracking Systems (ATS), candidates lack structured interview preparation, and no intelligent tool exists that maps a student's individual skill profile to real market demand while generating a personalised, **constraint-aware** learning path.

Existing career platforms are generic, Western-centric, and offer no adaptive AI reasoning. **Baymax.app** directly solves this gap by deploying a multi-agent AI system whose roadmap planner is implemented as a **classical CSP (Constraint Satisfaction Problem)** — fulfilling the AI-Lab requirement for a non-LLM, classical AI algorithm with a step-by-step visualization — surrounded by four supporting agents that handle resume analysis, voice interviewing, job search and plan summarisation.

2. Project Objectives — Achieved vs. Proposed

The proposal listed six concrete objectives. The table below records the final, delivered status of each.

#	Proposed objective	Status	Where it lives
1	Resume analysis vs. JD with ATS score (0–100) and structured skill-gap report	✅ Delivered	`backend/agents/resume_agent.py` · `frontend/src/components/ResumeAnalyzer.tsx`
2	Live, adaptive voice mock interview (Groq Whisper)	✅ Delivered with deterministic feature extraction + difficulty ramping	`backend/agents/interview_agent.py` · `frontend/src/components/InterviewCoach.tsx`
3	Live job listings ranked by skill match	✅ Delivered with deterministic Jaccard-overlap scoring + spam filter	`backend/agents/job_search_agent.py` · `frontend/src/components/JobScout.tsx`
4	**Personalised roadmap via Constraint Satisfaction Problem (CSP)**	✅ Delivered — AC-3 + Backtracking with MRV / LCV / Forward-checking	`backend/agents/csp_planner.py`
5	**Step-by-step visualization of the CSP** (variables, domains, constraint propagation, backtracking, assignments)	✅ Delivered — animated, scrubbable, speed-controllable	`frontend/src/components/CSPVisualizer.tsx`
6	Fully deployed, public-URL working system	✅ Live at `baymax-app-six.vercel.app` (Vercel + DigitalOcean)	`frontend/vercel.json` · `backend/Dockerfile`

A seventh, self-imposed objective was added late in development: strip the codebase of every component not relevant to the AI-Lab grade. The result is an 8-component frontend (down from 19) and a backend whose graded module — the CSP — is **completely free of LLM calls**.

3. System Implementation

3.1 Multi-Agent System Architecture

The system follows a **hierarchical pipeline orchestration** strategy. State is held in a single shared session object (`useUserSession`) on the frontend; the backend exposes purpose-built REST endpoints for each agent. Inputs are validated and prompt-injection-screened by a central `_sanitize` layer before reaching any LLM.

```
┌──────────────────────────────────────────────────────────────────────┐
│                              USER SESSION                            │
│              (resume text · job desc · skills · interview avg)       │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                │
   ┌────────────────────────────┼────────────────────────────┐
   ▼                            ▼                            ▼

 ┌──────────────┐           ┌──────────────┐                      ┌──────────────┐
 │  RESUME      │           │  INTERVIEW   │                      │  JOB SCOUT   │
 │  ANALYZER    │ ────────► │  COACH       │ ──────────────────►  │  (Serper +   │
 │  (Alex)      │ skills    │  (Sam)       │   weak areas         │  Jaccard     │
 │  Groq LLM    │ gaps      │  Groq + STT  │                      │  ranking)    │
 └──────────────┘           └──────────────┘                      └──────────────┘
        │                                                                  │
        │                                                                  │
        └──────────────────────────────────────────────────────────────────┘
                                │
                                ▼
                        ┌──────────────────┐
                        │  ROADMAP PLANNER │
                        │  ┌──────────────┐│
                        │  │ CSP SOLVER   ||  ← AI-Lab graded module
                        │  │ AC-3 + BT + MRV||    (LLM-free, deterministic)
                        │  │ + trace events||
                        │  └──────────────┘│
                        │  + LLM Plan Summary │  (optional, supplementary)
                        │  + free-first res.  │
                        └──────────────────┘
```

Each REST call is small and idempotent. The frontend keeps the canonical state in browser local-storage, so a refresh in the middle of a pipeline does not lose the candidate's progress.

3.2 Agent 1 — Resume Analyzer (Alex)

File: `backend/agents/resume_agent.py` , `backend/tools/pdf_tool.py`

Two distinct services live here:

1. **Heuristic structural parser** — `extract_structured_resume()` walks the extracted PDF text line-by-line, classifies each line as either a section heading (matched against ~20 hand-curated regex aliases) or content, and returns a payload of `{profile, summary, education, experience, skills, projects, certifications}` . This is **rule-based, deterministic and zero-LLM** — important for predictable behaviour and instant response.

2. **LLM analyzer** — `analyze_resume_structured()` sends the resume text and a target job description to Groq LLaMA 3.3 70B with a strict JSON schema, returning `overall_score`, `ats_score`, `match_score`, strengths, weaknesses, missing keywords and rewritten bullets.

Section-heading recognition example:

```
# backend/tools/pdf_tool.py
SECTION_ALIASES: dict[str, list[str]] = {
    "summary":    [r"summary", r"professional\s+summary", r"profile", r"objective",
r"about\s+me"],
    "education":  [r"education", r"academic\s+background", r"qualifications"],
    "experience": [r"experience", r"work\s+experience", r"professional\s+experience",
                   r"employment", r"employment\s+history"],
    "skills":     [r"skills", r"technical\s+skills", r"core\s+skills", r"technologies",
                   r"tech\s+stack", r"core\s+competencies"],
    "projects":   [r"projects", r"personal\s+projects", r"academic\s+projects"],
    ...
}
```

📷 **Screenshot:** Resume Analyzer with score cards · `docs/screenshots/01-resume-analyzer.png`

3.3 Agent 2 — Interview Coach (Sam)

File: `backend/agents/interview_agent.py`

Two ideas drive this module:

1. **Personalisation comes from explicit features, not from hoping the LLM reads the resume well.** Before every call we extract `{tech, projects, companies}` deterministically from the resume text and hand them to the LLM as bullet points it MUST reference. So instead of "Tell me about yourself", the model is forced to ask "In your *Baymax — AI Career Coach* project, how did you handle X?".
2. **Difficulty adapts to performance.** Every prior `"score": N` from the conversation history is parsed; the running average drives a hint that ramps the next question up or down.

```
# backend/agents/interview_agent.py — adaptive difficulty
running = sum(prior_scores) / len(prior_scores) if prior_scores else 6.0
if running < 5:
    difficulty_hint = "Drop one level — ask a simpler, scoped-down question on the same
topic."
elif running > 7.5:
    difficulty_hint = "Push harder — ask about trade-offs, scaling, or what they'd change
in hindsight."
else:
    difficulty_hint = "Stay at this level and move to the next topic."
```

Voice flow uses **Groq Whisper Large v3** for transcription on the backend and the browser's **Web Speech Synthesis API** for asking questions aloud (zero TTS cost, works offline).

📷 **Screenshot:** Voice interview in progress with the orb animation · `docs/screenshots/02-interview-`

> `coach.png`

3.4 Agent 3 — Job Scout (Zara)

File: `backend/agents/job_search_agent.py`

The previous version pushed raw Serper search hits straight into an LLM and asked it to rank them. The result was the classic *garbage in → garbage out* failure. The current pipeline is:

```
Serper queries  →  spam filter  →  deterministic score  →  top-K  →  LLM formats only
```

- **Spam filter** — drops Udemy/Coursera/blog/salary-calculator URLs and search-index pages.
- **Deterministic score** — `Jaccard(skills ∩ title+snippet) × 4 + domain_bonus + level_bonus`. Rozee.pk / Mustakbil / LinkedIn get bonus; "senior / lead / principal / manager" get a penalty (entry-level candidate).
- **Format-only LLM step** — the LLM is told it MUST use the URLs we passed in. A post-processor drops any line whose URL the LLM hallucinated outside the shortlist.

```
# backend/agents/job_search_agent.py
def _score_hit(hit: dict, candidate_skills: list[str]) -> float:
    domain  = _domain_of(hit.get("link", ""))
    overlap = _skill_overlap(candidate_skills, f"{hit.get('title','')}
{hit.get('snippet','')}")
    overlap_score = min(0.55, overlap * 4.0)
    domain_bonus  = max((b for d, b in DOMAIN_BONUS.items() if d in domain), default=0.0)
    level_bonus   = _level_score(hit.get("title",""), hit.get("snippet",""))
    return max(0.0, min(1.0, overlap_score + domain_bonus + level_bonus))
```

Test input	Score
"Junior Python Engineer at Folio3" — `rozee.pk` — Python/Django/AWS/Docker	**1.00**
"Trainee Software Engineer" — `mustakbil.com` — fresh graduate ML/React	**1.00**
"Senior Manager — Engineering" — `linkedin.com` — 10+ years	**0.00**

> 📷 **Screenshot:** Job Scout with ranked cards · `docs/screenshots/03-job-scout.png`

3.5 Agent 4 — Roadmap Planner (Rahul)

File: `backend/agents/csp_planner.py` + `backend/agents/career_planner_agent.py`

Two complementary services:

1. **CSP solver** — the graded AI algorithm. Fully covered in Section 4.
2. **Plan Summary (LLM)** — supplementary; produces a plain-English version of the schedule the CSP just computed. The frontend renders it side-by-side with the CSP visualization in a separate tab.

The frontend tab order is intentional: 🧠 **CSP Algorithm** is the default, 📋 **Plan Summary** is second, 📚 **Resources** is third. Resources are sorted **free** → **free w/ aid** → **paid** before display.

> 📷 **Screenshot:** Roadmap planner with three tabs · `docs/screenshots/04-roadmap-tabs.png`

4. Selected AI Algorithm – Constraint Satisfaction Problem (CSP)

The career-planning task is modelled as a CSP `⟨V, D, C⟩`. This is the classical AI technique chosen from the approved list and is the focus of grading.

4.1 Formulation

Component	Definition
Variables	One per learning task derived from the user's skill gaps (`Strengthen Python` , `Learn Docker` , `System Design Fundamentals` , …). Two synthetic variables — `capstone` and `applications` — are always added.
Domains	Each variable's domain is `{1, 2, …, N}` where `N` is the user-chosen planning horizon in weeks (default 12, range 4–26).
Constraints	(a) **Unary earliest** `week(X) ≥ e_X` (foundations early, advanced late); (b) **Unary deadline** `week(X) ≤ d_X` ; (c) **Binary prerequisite** `week(A) < week(B)` ; (d) **Binary exclusive** `week(A) ≠ week(B)` (avoid burn-out); (e) **N-ary workload** Σ hours per week ≤ B (default 15 h/week, range 4–60).

4.2 Stage 1 – Node Consistency

Every variable's unary constraints are applied first. If any domain wipes out, the solver fails fast with `reason = "unary_dead_end"` .

4.3 Stage 2 — AC-3 (Arc Consistency Algorithm 3)

```
# backend/agents/csp_planner.py — _ac3()
def _ac3(self) -> bool:
    queue: deque[tuple[str, str]] = deque(self.binary_pred.keys())
    while queue:
        xi, xj = queue.popleft()
        revised, removed = self._revise(xi, xj)
        if revised:
            self._log("ac3_revised",
                      f"AC-3: pruned {removed} from D({xi}). "
                      f"D({xi}) is now {self.domains[xi]}.",
                      variable=xi, removed_values=removed)
            if not self.domains[xi]:
                return False                      # domain wiped — inconsistent
            for xk in self.neighbors[xi]:         # re-enqueue affected arcs
                if xk != xj:
                    queue.append((xk, xi))
    return True
```

`_revise(xi, xj)` removes from `D(xi)` any value with no supporting partner in `D(xj)` according to the binary predicate stored at `binary_pred[(xi, xj)]`. Every prune produces a **trace event**, recorded in order with snapshots of `domains` and `assignment`.

4.4 Stage 3 — Backtracking Search (with MRV / LCV / Forward-Checking)

```
# backend/agents/csp_planner.py — _backtrack()
def _backtrack(self) -> bool:
    if len(self.assignment) == len(self.variables):
        return True
    var = self._select_unassigned_variable()      # MRV (+ degree, + insertion)
    for value in self._order_domain_values(var):  # LCV
        ok, reason = self._is_consistent(var, value)
        if not ok: continue
        self.assignment[var] = value
        saved_domain = list(self.domains[var])
        self.domains[var] = [value]
        fc_ok, removed = self._forward_check(var, value)
        if fc_ok and self._backtrack():
            return True
        self._restore(removed)                     # undo on failure
        self.domains[var] = saved_domain
        del self.assignment[var]
    return False
```

- **MRV** — pick the variable with the smallest current domain.
- **LCV** — for that variable, try values that rule out the *fewest* values in neighbours first.
- **Forward checking** — after each assignment, prune values from neighbour domains that are now inconsistent. Restore on backtrack.

- **Workload** — checked separately on each assignment because it is N-ary.

4.5 Trace Event Schema

Every solver step appends an event to `trace`:

```
{
  "step": 27,
  "type": "bt_assign",
  "description": "Backtracking: assigned 'Docker & Containers' → week 2.",
  "variable": "docker",
  "value": 2,
  "domains":    { "docker": [2], "react": [2,3,4,5,6,7,8,9], ... },
  "assignment": { "python": 1, "docker": 2 }
}
```

The frontend simply replays this list to animate the algorithm — meaning the visualisation is faithful to the actual solver run, not a re-creation.

4.6 Stats from a Default Run

Input: `["python", "docker", "react", "system design", "machine learning", "interview"]`, 12 weeks, 15 h/week.

Stat	Value
Variables (tasks)	8 (6 skills + capstone + applications)
Constraints	9 prerequisite + 3 exclusive + 1 global workload
Trace events emitted	**70**
AC-3 arc checks	**29**
Values pruned by AC-3	5
Backtracking assignments	8
Backtracks (undos)	0
Final schedule	W1 Python · W2 Docker+React · W3 Interview · W4 System Design · W5 ML · W11 Capstone · W12 Applications

For a deliberately over-constrained input (8 skills, 8 weeks, 10 h/week) the solver correctly reports `success=false, reason=bt_failed` after 295 backtracks — proving the search actually explores rather than always succeeding on the first guess.

4.7 Visualization

File: `frontend/src/components/CSPVisualizer.tsx`

Five things are rendered live, fulfilling the rubric's CSP visualization sub-points:

1. **Variables** — task cards on the left.
2. **Domains** — week-number chips inside each card; pruned values fade and strike through as AC-3 runs.
3. **Constraints** — explicit list with the active arc highlighted during AC-3.
4. **Backtracking** — current variable + value highlighted, conflict reasons shown, `bt_unassign` events animate the undo.
5. **Assignments** — the right-hand calendar fills in week-by-week as the search commits.

Step controls: ⏮ ◀ ▶ ▶▶ ⏭ + 0.5× / 1× / 2× / 4× speed selector, plus a step counter and progress bar.

> 📷 **Screenshot:** CSP visualizer mid-run, AC-3 step active · `docs/screenshots/05-csp-ac3-pruning.png`
> 📷 **Screenshot:** CSP visualizer with completed schedule · `docs/screenshots/06-csp-final-schedule.png`

5. User Interface Walkthrough

The Dashboard implements a strict five-step pipeline. Each step is gated by the prior — the user cannot jump to the Interview before they have either built a resume or completed an analysis.

Tab	Agent	What the user does
0 · Resume Builder	Cass	Upload a PDF (parsed with the structural parser) or build from scratch in an open-resume-style accordion. AI-enhance individual bullets. Live A4 print preview.
1 · Resume Analyzer	Honey (Alex)	Paste a JD. Get ATS score, strengths/gaps, missing keywords, AI-rewritten bullets, "Apply to Builder" buttons.
2 · Mock Interview	Hiro (Sam)	Voice interview. Browser TTS asks the question; user speaks; Whisper transcribes; LLM scores 0–10 and asks the next question, with difficulty ramping per Section 3.3.
3 · Job Scout	Fred (Zara)	Filtered, deterministically-ranked job cards from Rozee.pk / Mustakbil / LinkedIn / Indeed / Wellfound. Apply links go to the *exact* URL Serper returned — no hallucination.
4 · Career Roadmap	Rahul	Three sub-tabs: 🧠 **CSP Algorithm** · 📋 **Plan Summary** · 📚 **Resources** (free-first).

> 📷 **Screenshot:** Dashboard sidebar with progress bar · `docs/screenshots/07-dashboard.png`
> 📷 **Screenshot:** Resume Builder side-by-side with live preview · `docs/screenshots/08-resume-builder.png`

6. Tools & Technologies

Layer	Technology	Purpose
AI Algorithm (course req.)	**CSP — AC-3 + Backtracking + MRV / LCV / Forward-Checking**	Career-roadmap generation with full step-by-step trace
LLM inference	Groq API (LLaMA 3.3 70B)	Sub-second responses for Resume Analyzer, Interview Coach, Plan Summary
Speech-to-Text	Groq Whisper Large v3	Real-time voice-interview transcription
Text-to-Speech	Browser Web Speech Synthesis API	Reading interview questions aloud — zero external cost
Job search	Serper API (Google Search index)	Live listings from rozee.pk, mustakbil.com, linkedin.com, indeed.com, wellfound.com
Backend	**FastAPI + Python 3.11**	REST endpoints, agent orchestration, validation
Frontend	**React 18 + Vite + TypeScript + Tailwind**	Dashboard UI, CSP visualizer animations
Resume PDF parsing	`pypdf` (modern) → `PyPDF2` (legacy fallback)	Plain-text + heuristic structural parser
Persistence	localStorage (frontend); optional Mem0 (backend, gracefully degrades to in-process dict)	Per-user session continuity
Deployment	Vercel (frontend) · DigitalOcean App Platform (backend)	`https://baymax-app-six.vercel.app`
Version control	Git + GitHub (`taha-zaidii/baymax-app`)	Source code

The frontend is **441 KB JS / 75 KB CSS gzipped** in production. Backend cold start under 5 s. Median CSP solve time on the default input: **< 30 ms** (the bulk of perceived latency is the optional LLM Plan-Summary call).

7. Test Cases & Results

#	Scenario	Expected	Observed
1	CSP — 6 typical skills, 12 wk, 15 h/wk	success, schedule respects prereqs + budget	✅ 8 tasks scheduled, 0 backtracks
2	CSP — 8 skills, 8 wk, 10 h/wk (over-constrained)	failure with reason `bt_failed`	✅ 295 backtracks, false
3	CSP — unknown skills (`"cooking", "dancing"`)	fall back to default tasks, still solve	✅ 4 default tasks, success
4	CSP — empty `skills_gap`	HTTP 400	✅
5	CSP — `total_weeks=100`	HTTP 400 (out of allowed 4..26)	✅
6	Job filter — Udemy course URL	rejected as spam	✅
7	Job filter — Indeed search-index page	rejected as spam	✅
8	Job score — junior+rozee vs senior+linkedin (entry candidate)	junior outranks senior	✅ 1.00 vs 0.00
9	Resume parser — sample resume with named sections	name + email + 5 sections extracted	✅
10	Resume Analyzer — PDF upload then click "Analyze"	analysis renders, no black screen	✅ (regression test for the bug originally reported)
11	Interview features — text containing "JavaScript"	does NOT also match "Java"	✅ (word-boundary regex)
12	Interview features — "PROJECTS … SKILLS"	does NOT count "SKILLS" as a project name	✅ (heading blocklist)

End-to-end smoke tests are scripted in the README under "Verifying the build" so the grader can re-run them.

8. Application in the Real World

`baymax-app-six.vercel.app` is publicly accessible — **no setup required to evaluate**. Concrete real-world applicabilities:

- **Individual students** — a personal AI career coach used before placement season to maximise interview readiness and resume quality.
- **Universities (FAST NUCES, NUST, IBA)** — deployable as a campus-wide career-prep platform for thousands of students simultaneously; the CSP component is fully deterministic so per-user solve time stays <50 ms.
- **Bootcamps and training institutes** — benchmark student readiness and generate adaptive, constraint-scheduled learning paths.
- **Recruiters** — the deterministic ranking layer can be repurposed as a pre-screen for skill-verified candidate shortlists from Rozee.pk and LinkedIn.

The chosen algorithm — CSP — is a particularly good fit for the domain: every real-world learning plan is constrained by hours, prerequisites and deadlines. Modelling those as classical CSP constraints lets the system *prove* its schedule is feasible, something a pure LLM cannot do.

9. Limitations & Honest Caveats

- **Skill catalogue is curated.** The mapping from free-text skill gap → catalog entry (`SKILL_CATALOG` in `csp_planner.py`) is hand-written. An unfamiliar skill is matched by substring or falls back to a default. This makes the CSP demo deterministic and explainable, but the planner does not generalise to every conceivable skill name.
- **Workload constraint is enforced on assignment, not via AC-3.** Because AC-3 is defined on binary constraints, the N-ary weekly-hour budget is checked during backtracking. This is the textbook approach but means very tight budgets produce many backtracks before failing.
- **No optimality guarantee.** CSP returns the *first* feasible assignment. A preference for "earlier weeks first" is encoded only via LCV tie-breaking.
- **Auxiliary agents use LLMs.** The graded module — the CSP — contains zero LLM calls. The Resume Analyzer, Interview Coach, Job Scout (formatter step) and Plan Summary do call Groq LLaMA 3.3 70B; this is a deliberate scope decision (LLMs handle natural-language tasks where they shine; CSP handles the constraint-satisfaction task where they don't).
- **PDF text extraction is layout-lossy.** Multi-column resumes, heavy graphics, or non-Latin scripts can degrade the structural parser. The "raw-edit" fallback inside the Builder gives the user a way to recover.

10. Running the Project Locally

10.1 Prerequisites

Tool	Version
Python	3.10 + (3.11 / 3.12 tested)
Node.js	18 +
npm	9 +
Groq API key	from https://console.groq.com/keys
Serper API key (optional)	from https://serper.dev — only needed for Job Scout

10.2 Backend

```
cd backend
python3 -m venv venv
source venv/bin/activate            # Windows: .\venv\Scripts\activate
pip install -r requirements.txt
cp ../.env.example ../.env          # add GROQ_API_KEY (and optionally SERPER_API_KEY)
uvicorn api:app --reload --port 8000
```

The API will be live on `http://localhost:8000`. Open `http://localhost:8000/docs` for the auto-generated Swagger UI.

10.3 Frontend

```
cd frontend
npm install
npm run dev                          # starts Vite on http://localhost:5173
```

Vite proxies `/health`, `/resume/*`, `/interview/*`, `/jobs`, `/roadmap/*` to `localhost:8000` automatically — no extra config needed.

10.4 Verifying the build

```
# From baymax.app/backend
source venv/bin/activate
python -c "
from fastapi.testclient import TestClient
from api import app
c = TestClient(app)

print(c.get('/health').status_code)                      # → 200

r = c.post('/roadmap/csp', json={
    'skills_gap': ['python','docker','react','system design','machine
learning','interview'],
    'total_weeks': 12, 'weekly_hour_budget': 15})
print(r.status_code, r.json()['success'], r.json()['stats'])
"
```

```
# From baymax.app/frontend
npm run build                        # production bundle in dist/
npx tsc --noEmit -p tsconfig.app.json   # zero errors expected
```

10.5 Demo flow for the grader

1. Open the Dashboard.
2. **Tab 0 · Resume Builder** — upload any PDF resume *or* type into the form.
3. **Tab 1 · Resume Analyzer** — paste a job description, click *Analyze Resume*.
4. **Tab 2 · Mock Interview** — click *Start*; speak your answers; observe the difficulty ramp.
5. **Tab 3 · Job Scout** — observe ranked, deterministically-scored job cards.
6. **Tab 4 · Career Roadmap** → 🧠 **CSP Algorithm** — click *Run CSP Solver*, then *Play* on the controls. Watch the algorithm step through (the graded part).

> 📷 **Screenshot:** the full demo flow side-by-side · `docs/screenshots/09-demo-flow.png`

11. Submission Checklist

- [x] Working UI (React + Tailwind, deployed)
- [x] Algorithm logic — full CSP solver in `backend/agents/csp_planner.py` (520 lines, fully docstring'd)
- [x] Visual representation — animated step-through in `frontend/src/components/CSPVisualizer.tsx`
- [x] CSP visualization sub-points — variables ✓ · domains ✓ · constraints ✓ · backtracking ✓ · assignments ✓
- [x] Clean, well-commented code (top-of-file design notes on every new file; inline comments where the *why* is non-obvious)

- [x] Project report — this document
- [x] Source code in the repository (`taha-zaidii/baymax-app`)
- [x] Live demo accessible at https://baymax-app-six.vercel.app
- [ ] Screenshots placed in `docs/screenshots/` — see `docs/screenshots/README.md` for the capture checklist
- [ ] Demo video recorded and linked

Appendix A — Repository Map

```
baymax.app/
├── PROJECT_REPORT.md                    ← this file
├── README.md                            ← course-quick-reference + run instructions
├── backend/
│   ├── api.py                           FastAPI routes (CSP, Resume, Interview, Jobs,
Roadmap)
│   ├── config.py                        API-key loading + responsible-AI flags
│   ├── agents/
│   │   ├── csp_planner.py               ★ AI-Lab algorithm: CSP / AC-3 / Backtracking
│   │   ├── resume_agent.py              Alex — analyzer + builder helpers
│   │   ├── interview_agent.py           Sam — feature extraction + adaptive interview
│   │   ├── job_search_agent.py          Zara — Serper + spam filter + Jaccard scoring
│   │   ├── career_planner_agent.py      Rahul — LLM Plan Summary + curated free
resources
│   │   └── memory_agent.py              Mem0 (graceful degradation)
│   └── tools/
│       ├── pdf_tool.py                  Plain-text + structural PDF parser
│       └── search_tool.py               Serper API wrapper
├── frontend/
│   └── src/
│       ├── pages/Index.tsx              Slim academic header + Dashboard
│       ├── components/
│       │   ├── Dashboard.tsx            5-tab pipeline shell
│       │   ├── ResumeBuilder.tsx        PDF→form parser + live A4 preview
│       │   ├── ResumeAnalyzer.tsx       ATS score + bullet rewriter
│       │   ├── InterviewCoach.tsx       Voice interview (TTS + Whisper)
│       │   ├── JobScout.tsx             Ranked job cards
│       │   ├── RoadmapPlanner.tsx       🧠 CSP / 📋 Summary / 📚 Resources
│       │   └── CSPVisualizer.tsx        ★ AI-Lab visualization
│       ├── hooks/use-user-session.ts    Central session store
│       └── lib/api.ts                   Typed API client
└── docs/screenshots/                    ← screenshots referenced in this report
```

The ★-marked files are the canonical entry points for grading.

Appendix B — REST API Reference

Method	Path	Purpose
GET	`/health`	Liveness probe
POST	`/extract-resume`	Plain text from a PDF
POST	`/resume/parse`	Structured `{profile, sections, ...}` from a PDF
POST	`/resume/analyze`	JSON resume + JD → analysis
POST	`/resume/analyze/upload`	PDF + JD → analysis (returns extracted text too)
POST	`/resume/improve`	Rewrite a single bullet
POST	`/resume/improve-section`	Rewrite a whole section
POST	`/resume/generate-section`	Generate a section from minimal context
POST	`/interview/start`	Begin a multi-turn voice interview
POST	`/interview/reply`	Submit an answer; get scored + next question
POST	`/interview/transcribe`	Whisper STT
POST	`/jobs`	Filtered + ranked job listings
POST	`/roadmap/csp`	**★ CSP solver — returns assignment, trace, stats**
POST	`/roadmap`	LLM Plan Summary
POST	`/roadmap/certifications`	Free-first curated resources for skill gaps

Submitted by Group BCS-6B — Taha Zaidi (group leader), Amna Khan, Kissa Zehra, Aiza Gazyani.
FAST NUCES Karachi — Spring 2026.